CV Sorting using Large Language Models: An Automated Candidate Ranking System

Abstract

This project aims to automate the screening and ranking of candidate resumes against specific job requirements using Large Language Models (LLMs). The system processes a Job Description (JD) and multiple CVs in PDF and DOCX formats and evaluates candidates based on skills, professional experience and education.

A two-LLM architecture is used with distinct roles. **Qwen2.5-1.5B-Instruct** converts unstructured JD and CV text into structured JSON for downstream evaluation, while **Qwen2.5-3B-Instruct** performs semantic evaluation of professional experience against the JD. Rule-based matching is used for skills and education, and the component scores are combined to generate an overall score and recommendation.

Candidates are ranked and the results are exported in CSV and JSON formats. Sequential model loading and memory clean-up are used to support execution in resource-constrained environments. The project demonstrates the meaningful integration of two lightweight LLMs to combine structured information extraction with semantic candidate evaluation

1. Introduction & Problem Statement

Recruitment requires reviewing candidate resumes against specific job requirements. Manual screening can be time-consuming and inconsistent, while traditional automated approaches often rely heavily on keyword matching and may fail to identify relevant experience expressed using different terminology.

This project develops an LLM-based CV sorting system that processes multiple CVs against a Job Description, evaluates skills, education and professional experience, and generates a ranked list of candidates. The use of LLMs enables contextual and semantic evaluation beyond direct keyword comparison.

2. Objectives

The objectives of the project are:

- Process JD and CV documents in PDF and DOCX formats.
- Convert unstructured documents into structured JSON using an LLM.
- Evaluate candidate skills and educational qualifications against the JD.
- Perform semantic evaluation of professional experience using a second LLM.

- Calculate overall candidate scores and recommendations.
- Rank candidates and export results in CSV and JSON formats.
- Implement the solution using lightweight locally executed LLMs.

3. Methodology

The solution is implemented in Python using libraries including PyTorch, Hugging Face Transformers, PyPDF, python-docx, Pandas and PyTesseract. The application is executed through a `main.py` entry point and accepts folder paths for the JD, CV documents and Output location.

The workflow begins by extracting text from the JD and CV documents. PDF and DOCX files are processed using appropriate document extraction libraries, while OCR can be used when required for documents without a usable text layer.

A two-LLM architecture is used with distinct responsibilities. **Qwen2.5-1.5B-Instruct (LLM1)** converts the unstructured JD and CV text into structured JSON containing information such as skills, education, professional experience and responsibilities. This structured information is then used for deterministic skill and education matching.

Qwen2.5-3B-Instruct (LLM2) performs semantic evaluation of professional experience. It compares job titles, work history, responsibilities, projects, technical context and domain relevance against the JD. The model is instructed to consider semantic equivalence while avoiding incorrect interpretation of unrelated business terminology as AI or machine learning experience.

Thus, both LLMs are meaningfully integrated into the workflow with distinct responsibilities: LLM1 performs unstructured document-to-structured-data conversion, while LLM2 performs semantic professional experience evaluation.

The component scores are combined using predefined weights to calculate the overall candidate score:

Overall Score = 50% Skill Match + 30% Experience Match + 20% Education Match

Candidates are subsequently ranked according to their overall scores and assigned recommendations. The final results are exported in CSV and JSON formats.

To manage computational resources, the two models are loaded sequentially. LLM1 is released after completing document structuring before LLM2 is loaded for semantic evaluation. Garbage collection and CUDA memory clean-up are performed between stages to reduce memory consumption.

4. Results and Analysis

The system was tested using CVs with varying relevance to an AI Engineer Job Description. The results differentiated candidates with relevant AI and machine learning experience from candidates in unrelated domains.

The output includes candidate name, CV filename, component scores, overall score, recommendation, matched and missing skills, and experience-based reasoning. Candidates are ranked in descending order based on their overall scores.

Testing also highlighted important model and implementation trade-offs. Larger models provided stronger semantic capabilities but required significantly more GPU memory, making them difficult to use within the available system constraints. Smaller models were more resource-efficient but occasionally produced inaccurate interpretations or unreliable outputs. This was mitigated through carefully designed task-specific prompts, explicit evaluation rules, structured JSON output requirements, and deterministic scoring where appropriate. Testing also identified practical implementation challenges. Longer multi-page CVs initially produced truncated LLM responses when the generation limit was insufficient, resulting in invalid JSON. Increasing the maximum generated tokens to 2000 resolved this issue. GPU memory limitations were also encountered when multiple models remained loaded simultaneously. Sequential model loading and memory clean-up were therefore incorporated into the final implementation.

.

The results demonstrate that combining deterministic matching with semantic LLM-based evaluation provides a more comprehensive assessment than relying solely on keyword matching.

5. Conclusion and Future Work

This project demonstrates an automated CV sorting and candidate ranking system using two LLMs with distinct roles: LLM1 converts JD and CV documents into structured information, while LLM2 performs semantic evaluation of professional experience. Combined with skill and education matching, the system generates ranked candidate results suitable for resource-constrained environments.

Future enhancements may include larger-scale testing, configurable scoring weights, improved scanned-document processing, and a web-based user interface.